Soal LKS DIY Bidang Cloud Computing 2026

Repository Soal

`https://github.com/lksdiycc/lksdiycc2026`

Fork repository di atas sebagai titik awal pengerjaan. Semua task dikerjakan berdasarkan kode yang tersedia di repository tersebut.

Skenario Utama: "Intelligent Streaming Content Recommendation API"

Latar Belakang

Sebuah perusahaan platform streaming bernama Streamify membutuhkan sistem rekomendasi dan forecasting konten berbasis Machine Learning. Sistem ini memprediksi kemungkinan seorang pengguna akan menonton/mendengarkan suatu konten, serta memperkirakan jumlah stream di masa mendatang.

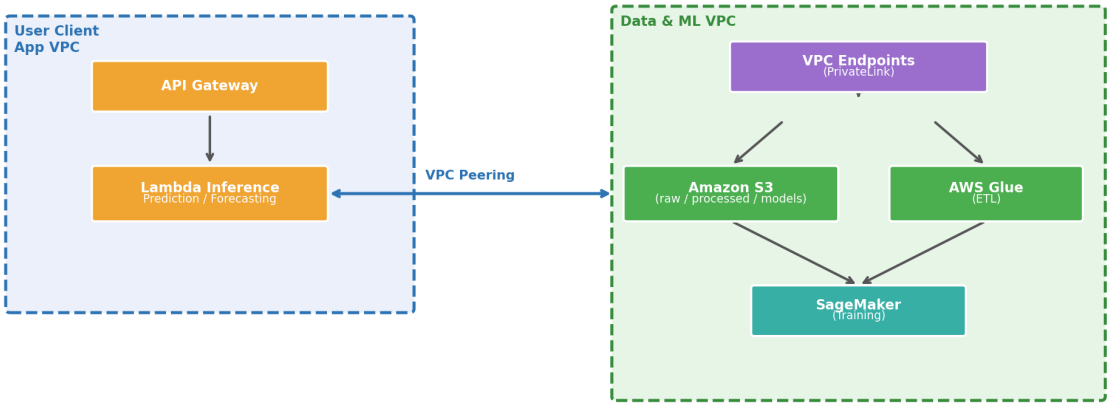
Seluruh layanan dibangun di atas AWS, dikemas dalam container Docker, dan di-deploy secara otomatis melalui pipeline CI/CD berbasis GitHub Actions. Agar sistem tetap andal saat terjadi gangguan, perusahaan membutuhkan mekanisme Automated Incident Response berbasis Generative AI yang dapat mendeteksi error pada Lambda, menganalisis log menggunakan LLM, dan mengirimkan laporan insiden otomatis melalui Amazon SNS.

Catatan: Seluruh pengerjaan berbasis repository **https://github.com/lksdiycc/lksdiycc2026**. Fork repository tersebut dan kerjakan sesuai instruksi di setiap bagian.

Waktu Pengerjaan: 7 Jam

Arsitektur Sistem

Diagram 1: Secure Data & ML Pipeline




```
User Client (App VPC)
  └── API Gateway (HTTPS)
        ├── Lambda: Stream Prediction    ← load model dari S3, query DynamoDB
        └── Lambda: Stream Forecasting  ← load model dari S3, query DynamoDB

EC2 / Container
  ├── lm-frontend (Go HTTP Server)      ← melayani UI, proxy ke API Gateway
  └── llm-integrate-sns (FastAPI)       ← webhook SNS, query CloudWatch Logs,
                                           kirim ke LLM, publish kembali ke SNS

AWS Services
  ├── S3          ← raw-data/ dan processed-data/ (model + dataset)
  ├── Glue        ← Crawler + ETL job (PySpark)
  ├── DynamoDB    ← user features, content features, stream history
  ├── CloudWatch  ← alarm error Lambda
  └── SNS         ← notifikasi email juri + trigger webhook
```

Dataset

Dataset tersedia di repository (dataset/dataset.py) dan menghasilkan 4 file CSV:

File	Jumlah Record	Deskripsi
user_profiles.csv	1.000	user_id, subscription_plan, total_streams, customer_segment, dll
content_catalog.csv	500	content_id, title, content_type, genre, avg_rating, is_exclusive, dll
user_interactions.csv	10.000	user_id, content_id, interaction_type

		(play/complete/like/skip/add_to_watchlist), watch_duration_seconds
subscription_history.csv	~750	plan (Basic/Standard/Premium), billing_period, amount, status

Struktur Repository (Target Akhir)

```
repo/
├── .github/workflows/ci.yml              ← GitHub Actions CI/CD
├── dataset/
│   ├── dataset.py
│   └── requirements.txt
├── ETL/sparks.py                          ← AWS Glue PySpark job
├── lambda/
│   ├── lambda_prediction/
│   │   ├── lambda_function.py
│   │   └── .env.example
│   └── lambda_forecasting/
│       ├── lambda_function.py
│       └── .env.example
├── lm-frontend/                           ← [DIBUAT SISWA]
│   ├── main.go  go.mod  Dockerfile  .env.example
│   └── html/  (index.html, prediction.html, forecasting.html)
├── llm-integrate-sns/                     ← [DIBUAT SISWA]
│   ├── app.py  requirements.txt  Dockerfile  .env.example
└── machine_learning/training.ipynb
```

Bagian 1: Data & ML Pipeline (Target: 2 Jam)

1.1 — S3 Data Lake

Buat S3 bucket dengan struktur folder berikut:

```
s3://[nama-bucket]/
├── raw-data/
│   ├── user-profiles/
│   ├── content-catalog/
│   ├── user-interactions/
│   └── subscription-history/
└── processed-data/
    ├── user_content_matrix/
    ├── content_stats/
    └── user_features/
```

Upload file CSV hasil dataset/dataset.py ke folder raw-data/ yang sesuai.

1.2 — AWS Glue ETL

- Buat Glue Crawler yang membaca seluruh raw-data/ dan mendaftarkan skema ke Glue Data Catalog (database: streamify_database).
- Buat Glue Job menggunakan ETL/sparks.py. Script membaca tabel user_profiles, user_interactions, content_catalog lalu menghasilkan: user_content_matrix, content_stats, dan user_features.
- Output disimpan ke processed-data/ dalam format Parquet.

1.3 — SageMaker / Model Training

- Buka SageMaker Notebook dan gunakan machine_learning/training.ipynb.
- Latih model hybrid recommender menggunakan data dari processed-data/.
- Simpan model artifact ke S3: s3://[nama-bucket]/models/hybrid_model.pkl

1.4 — Lambda Deployment

Deploy dua Lambda function dari folder lambda/:

Lambda Prediction

Variable	Nilai
MODEL_BUCKET	nama bucket S3
MODEL_KEY	models/hybrid_model.pkl
USERS_TABLE	streamify-users
CONTENT_TABLE	ContentEmbeddings

Lambda Forecasting

Variable	Nilai
FORECASTING_MODEL_BUCKET	nama bucket S3
FORECASTING_MODEL_KEY	models/forecasting_model.pkl
CONTENT_EMBEDDINGS_TABLE	ContentEmbeddings
STREAM_HISTORY_TABLE	StreamHistory
USER_INTERACTIONS_TABLE	UserInteractions

1.5 — API Gateway

- Buat API Gateway dan hubungkan: POST /predict → Lambda Prediction, POST /forecasts → Lambda Forecasting.
- Aktifkan API Key dan gunakan sebagai header x-api-key pada setiap request.

Bagian 2: Frontend & CI/CD (Target: 2 Jam)

2.1 — Go Frontend (lm-frontend/)

Buat HTTP server menggunakan Go (main.go) dengan spesifikasi:

- Melayani HTML statis: GET / → index.html, GET /prediction → prediction.html, GET /forecasting → forecasting.html
- Reverse proxy: POST /api/predict → API_PREDICTION; POST /api/forecast → API_FORECAST
- Inject header x-api-key dari env var API_GATEWAY_KEY pada setiap request ke API Gateway.

Environment variables (lm-frontend/.env.example):

```
API_PREDICTION=https://[id].execute-api.[region].amazonaws.com/[stage]/predict
API_FORECAST=https://[id].execute-api.[region].amazonaws.com/[stage]/forecasts
API_GATEWAY_KEY=[api-key]
PORT=3000
```

Buat Dockerfile untuk lm-frontend/ menggunakan base image golang dan expose port 3000.

2.2 — LLM Incident Response Service (llm-integrate-sns/)

Buat FastAPI webhook service (app.py) dengan endpoint POST /webhook:

1. SubscriptionConfirmation — konfirmasi otomatis via HTTP GET ke SubscribeURL.
2. Notification — ekstrak AlarmName, baca 5 log error terbaru dari CloudWatch Logs (filter "ERROR", 1 jam terakhir).
3. Kirim log ke LLM: "Sebagai DevOps, berikan 1 ringkasan penyebab error (Summary) dan 1 rekomendasi (Solusi) dari semua log berikut."
4. Publish hasil LLM ke SNS Topic (SNS_TOPIC_ARN) dengan subject: "Resume Incident Report: [AlarmName]".

LLM Provider (dikontrol via LLM_PROVIDER):

- ollama — POST ke OLLAMA_ENDPOINT dengan payload {model, prompt, stream: false}
- groq — POST ke https://api.groq.com/openai/v1/chat/completions menggunakan GROQ_API_KEY

Environment variables (llm-integrate-sns/.env.example):

```
LLM_PROVIDER=groq
OLLAMA_ENDPOINT=http://localhost:11434/api/generate
OLLAMA_MODEL=qwen2.5:1.5b
GROQ_API_KEY=xxxx
GROQ_MODEL=llama3-8b-8192
SNS_TOPIC_ARN=arn:aws:sns:[region]:[account]:NotificationMail
LIST_SNS_TOPIC_ARN={"ForecastingError":"/aws/lambda/Forecasting","PredictionError
":"/aws/lambda/Prediction"}
AWS_REGION=[region]
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
PORT=8080
```

Buat Dockerfile untuk llm-integrate-sns/ menggunakan base image python:3.12-slim.

2.3 — GitHub Actions CI/CD (.github/workflows/ci.yml)

Buat workflow yang berjalan saat push ke branch main dengan tiga jobs paralel:

Job 1 — build-frontend: Build Docker image dari lm-frontend/, push ke GHCR: ghcr.io/[owner]/[repo]:frontend-[timestamp]

Job 2 — build-llm-integrate-sns: Build Docker image dari llm-integrate-sns/, push ke GHCR: ghcr.io/[owner]/[repo]:llm-webhook-[timestamp]

Job 3 — build-example-dataset-csv: Setup Python 3.12, jalankan dataset/dataset.py, buat struktur raw-data/, zip menjadi raw-data.zip, upload semua CSV + zip ke GitHub Release dengan tag dataset.

Versioning image menggunakan format timestamp: yyyy-mm-dd-hh-mm-ss

Bagian 3: Monitoring & Automated Incident Response (Target: 3 Jam)

Diagram 2: Generative AI Incident Response Flow

3.1 — DynamoDB Tables

Buat tabel DynamoDB berikut dan populate dengan data dari CSV:

Tabel	Partition Key	Deskripsi
streamify-users	user_id (String)	Profil pengguna untuk Lambda Prediction
ContentEmbeddings	content_id (String)	Fitur konten untuk Lambda Prediction & Forecasting
StreamHistory	content_id (String)	Histori stream per konten untuk Lambda Forecasting
UserInteractions	user_id (String)	Log interaksi pengguna

3.2 — CloudWatch Alarms

Buat 2 CloudWatch Alarms:

Alarm	Lambda Target	Threshold	Action
PredictionError	Lambda Prediction	> 0 error dalam 1 menit	Publish ke SNS NotificationMail
ForecastingError	Lambda Forecasting	> 0 error dalam 1 menit	Publish ke SNS NotificationMail

3.3 — Amazon SNS

- Buat SNS Topic bernama NotificationMail.
- Tambahkan Email subscription ke alamat email juri.
- Tambahkan HTTPS subscription ke webhook llm-integrate-sns: http://[EC2-IP]:8080/webhook
- Service llm-integrate-sns akan otomatis mengkonfirmasi SubscriptionConfirmation.

3.4 — Deploy Layanan di EC2

Deploy container dari GHCR di EC2:

```
# Frontend
docker run -d -p 3000:3000 \
  -e API_PREDICTION=https://[...] \
  -e API_FORECAST=https://[...] \
  -e API_GATEWAY_KEY=[key] \
  ghcr.io/[owner]/[repo]:frontend-[version]

# LLM Webhook
docker run -d -p 8080:8080 \
  -e LLM_PROVIDER=groq \
  -e GROQ_API_KEY=[key] \
  -e SNS_TOPIC_ARN=arn:aws:sns:[...] \
  -e LIST_SNS_TOPIC_ARN='{"ForecastingError":"/aws/lambda/Forecasting"}' \
  -e AWS_ACCESS_KEY_ID=[...] \
  -e AWS_SECRET_ACCESS_KEY=[...] \
  ghcr.io/[owner]/[repo]:llm-webhook-[version]
```

3.5 — Pengujian End-to-End

1. Uji Prediction API — akses http://[EC2-IP]:3000/prediction, masukkan user_id dan content_id, verifikasi response berisi prediction_percentage.
2. Uji Forecast API — akses http://[EC2-IP]:3000/forecasting, masukkan content_id dan jumlah hari, verifikasi response berisi array forecast.
3. Uji Incident Response — picu error Lambda (kirim payload salah), tunggu CloudWatch Alarm aktif, verifikasi email berisi ringkasan AI masuk ke juri.

Kriteria Penilaian

NO	Aspek	Bobot
1	Dapat menggunakan Dataset dan GitHub (dataset.py, CI/CD, GitHub Release)	10
2	Pemrograman Python (Lambda function, FastAPI, dataset generator)	10
3	Konfigurasi VPC, Security Group, IAM Role, Subnetting	10
4	Konfigurasi Amazon Elastic Cloud Compute (EC2)	10
5	Integrasi database menggunakan Amazon DynamoDB	10
6	Konfigurasi Amazon S3 (Data Lake raw-data & processed-data)	10
7	Konfigurasi lm-frontend (Go HTTP Server & Reverse Proxy ke API Gateway)	10
8	Containerisasi layanan dengan Docker & push image ke GHCR	10
9	Konfigurasi AWS PrivateLink (VPC Endpoints)	10
10	Konfigurasi API Gateway (endpoint + API Key)	10
11	Lambda Function (Prediction & Forecasting)	10
12	AWS Glue (Crawler + ETL Job PySpark)	10
13	SageMaker dan penggunaan algoritma yang ada di dalamnya	10
14	Konfigurasi CloudWatch Alarms (monitoring Lambda error)	10
15	Amazon SNS (topik, email subscription, HTTPS webhook)	10
16	CI/CD GitHub Actions (build image + publish dataset ke Release)	10
17	Ollama/LLM dan proses integrasi incident response (llm-integrate-sns)	20
18	Integrasi dalam satu workflow + pertanyaan dewan juri	70
	Total Nilai	**250**

Dokumen ini adalah soal resmi LKS DIY Bidang Cloud Computing 2026.
Dilarang disebarluaskan sebelum hari pelaksanaan.